DOCUMENTO DE ESPECIFICAÇÃO DE REQUISITOS E ARQUITETURA SEGURA (SSD)

Laboratório de Inovação III — Prof. Edilberto Silva — 2026 versão 16.3

1. METADADOS DO PROJETO E DA EQUIPE

1.1 Identificação

NOME_DO_PROJETO: Sistema ERP Construção DESCRICAO_BREVE: Módulo de gestão de recuperação de vendas, permitindo filtragem de orçamentos pendentes e contato direto via WhatsApp com registro de autoria.

1.2 Localização dos Artefatos (URLs Obrigatórias)

LINK_REPOSITORIO_GITHUB: https://github.com/Jotajhones/sistema_ERP_LI3_FACSENAC LINK_APLICACAO_DEPLOY: https://sistema-erp-li3-facsenac.onrender.com/ LINK_BANCO_DADOS: https://supabase.com/dashboard/project/qlfjdnljfmicjkrbrwih/editor/ LINK_DEMONSTRAÇÃO: https://sistema-erp-li-3-facsenac.vercel.app/

1.3 Composição da Equipe

ID	Nome Completo	Papel Primário	Papel Secundário	E-mail / Contato
1	João Pedro de Lima	Product Owner	Tech Lead / DevOps	joao52517876@edu.df.senac.br
2	Miguel Nunes	Desenvolvedor Back-End	Full Stack	miguel59970766@edu.df.senac.br
3	Luciano Santos	Desenvolvedor Back-End	Full Stack	luciano58995256@edu.df.senac.br
4	Daniel Silva	DBA / Banco de Dados	Full Stack	daniel52370026@edu.df.senac.br
5	Arthur Andrade	Desenvolvedor Front-End	Full Stack	arthur59275206@edu.df.senac.br
6	Pedro Soares	Desenvolvedor Front-End	Full Stack	pedro59153296@edu.df.senac.br
7	Matheus Chagas	AppSec / QA	Full Stack	matheus57119256@edu.df.senac.br

RF-007: Recuperação de Orçamentos via Integração WhatsApp

1. Identificação

- **ID:** RF-007
- **Título:** Recuperação de Orçamentos Pendentes e Tracking de Contato
- **Prioridade:** Alta
- **Complexidade:** 5 story points

a) URL da Aplicação (Deploy Online)

- **URL:** https://sistema-erp-li-3-facsenac.vercel.app/rf-007
- **Plataforma:** Vercel
- **Acesso:** Público

b) Link do Banco de Dados Online

- **BD:** Supabase (PostgreSQL)
- **Link:** https://supabase.com/dashboard/project/qlfjdnljfmicjkrbrwih/editor/
- **Acesso:** Leitura para o professor (email: prof.edilberto.silva@gmail.com)
- **Tabela principal:** orcamentos_contatos

c) Link do GitHub (Repositório)

- **Repositório:** https://github.com/Jotajhones/sistema_ERP_LI3_FACSENAC
- **Branch principal:** main
- **Estrutura:** Conforme guia (docs/, src/, database/)

---

2. Descrição e Atores

O requisito de Recuperação de Orçamentos permite que os vendedores do sistema ERP filtrem proativamente orçamentos que foram gerados, mas ainda não convertidos em Ordens de Serviço (vendas reais). Através de uma integração leve via link direto, o sistema aciona o WhatsApp Web/Desktop do operador, pré-formatando uma mensagem de resgate da venda.

Para garantir a rastreabilidade sem inflar as tabelas financeiras, o sistema cria um registro em uma tabela associativa assim que o vendedor retorna à tela do ERP. Isso garante que a gestão saiba quais orçamentos foram acompanhados e qual vendedor realizou o esforço de recuperação, evitando que o cliente seja abordado múltiplas vezes pelo mesmo motivo.

Benefícios para o Negócio

- Aumentar a taxa de conversão de orçamentos abandonados através de contato proativo estruturado.
- Evitar conflito comercial, impedindo que dois vendedores entrem em contato com o mesmo cliente pelo mesmo orçamento.
- Gerar métricas de engajamento da equipe comercial para acompanhamento gerencial.

Atores

Vendedor

- **Papel:** Operador comercial responsável por resgatar a venda e contatar o cliente.
- **Permissões:** Read (orçamentos pendentes), Create (registro de contato).
- **Acesso:** Tela de histórico de orçamentos.

Gestor / Admin

- **Papel:** Auditoria de desempenho da equipe comercial.
- **Permissões:** Read, Delete (excluir registro de contato caso marcado por engano, visando liberar o cliente para novo contato).
- **Acesso:** Dashboard e relatórios globais.

Sistema (Automático)

- **Papel:** Formatador e Registrador.
- **Permissões:** Validação do token do vendedor, montagem do URI E.164 e registro de timestamps na base.

Validações Necessárias

- O contato só pode ser registrado se o orçamento estiver com status estritamente igual a PENDENTE.
- O contato não pode ser registrado se o cliente do orçamento não possuir telefone cadastrado.
- O ID do vendedor deve ser inferido do Token de sessão (JWT/UUID), e não do corpo da requisição.

---

3. Casos de Uso e Requisitos Não-Funcionais

Fluxo Principal: Contatar Cliente e Registrar Ação

Pré-condições:

- Usuário autenticado com perfil VENDEDOR ou GESTOR.
- Cliente vinculado ao orçamento possui telefone válido.
- Orçamento possui status PENDENTE no banco de dados.

Fluxo:

1. Vendedor acessa a página de Histórico de Orçamentos.
2. Vendedor seleciona o filtro "Não Contatados".
3. Sistema consulta o backend que realiza o cruzamento de dados, retornando apenas orçamentos sem registro na tabela de contatos.
4. Sistema exibe a lista filtrada na tela com ícone de WhatsApp.
5. Vendedor clica no ícone de contato de um orçamento específico.
6. Sistema extrai nome do cliente, nome do vendedor, telefone e valor.
7. Sistema higieniza o telefone (apenas números) e codifica a mensagem de saudação.
8. Sistema abre nova aba direcionando para o serviço wa.me.
9. Vendedor envia a mensagem e retorna o foco para a aba do sistema ERP.

10. Sistema detecta o retorno de foco e dispara a rota POST para registrar o contato.
11. Sistema salva a ação no banco de dados vinculando ao ID do vendedor.
12. Sistema atualiza a linha na tabela visualmente com a tag "Contatado".

Pós-condições:

- Registro único criado na tabela orcamentos_contatos.
- Orçamento removido dinamicamente da visão "Não Contatados".
- Ação vinculada de forma inalterável ao ID do usuário autenticado.

Fluxos Alternativos

Fluxo Alternativo A1: Cliente sem Telefone

1. No passo 6, o sistema detecta ausência de telefone.
2. Sistema exibe alerta: "Cliente não possui telefone cadastrado. Ação cancelada."
3. Fluxo é abortado e o link externo não é aberto.

Fluxo Alternativo A2: Orçamento já convertido

1. No passo 10, o backend verifica o status real do orçamento no banco.
2. Backend identifica status FINALIZADA ou CONVERTIDO.
3. Backend retorna erro HTTP 409 Conflict.
4. Sistema avisa o usuário que a venda já foi concretizada e atualiza a tabela.

Fluxo Alternativo A3: Duplo Clique (Idempotência)

1. No passo 10, o frontend dispara duas requisições simultâneas.
2. Backend tenta registrar a segunda.
3. Constraint UNIQUE da tabela orcamentos_contatos bloqueia a segunda inserção.
4. Backend trata a exceção e retorna sucesso silencioso para evitar erros na interface.

Regras de Negócio

- **RN-01:** Limite de Estado: O registro de contato só é aceito se a tabela orcamentos atestar status PENDENTE.
- **RN-02:** Formatação Internacional: O telefone repassado na URL deve obrigatoriamente possuir o DDI 55 inserido pelo frontend.
- **RN-03:** Auditoria Transparente: O identificador de quem fez o contato deve vir obrigatoriamente da assinatura do token de autenticação HTTP.
- **RN-04:** Restrição de Alteração: Vendedores não possuem autorização para utilizar o método DELETE no registro de contato.
- **RN-05:** Reaproveitamento de Interface: A ação ocorre nativamente na listagem existente, sem necessidade de navegação para novas páginas de CRM.
- **RN-06:** Prevenção de Perda de Foco: A navegação para a plataforma externa do WhatsApp deve obrigatoriamente ocorrer em uma nova aba do navegador.

Requisitos Não-Funcionais

- **RNF-01:** Usabilidade: O link para o WhatsApp deve abrir utilizando target="_blank".

- **RNF-02:** Performance: O cruzamento de dados (JOIN lógico) para filtrar pendentes deve responder em menos de 800 milissegundos.
- **RNF-03:** Segurança: Uso obrigatório da função encodeURIComponent do Javascript para prevenir injeção no parâmetro GET da URL externa.
- **RNF-04:** Escalabilidade: O banco de dados deve possuir índices na chave estrangeira orcamento_id para suportar consultas pesadas no futuro.

---

4. Protótipo Funcional e Integração de API

> ** AVISO DE SEGURANÇA E PROTEÇÃO GLOBAL DE ROTAS (A01:2021)**
>
> Faz-se necessário explicitar que **absolutamente todas as rotas do backend desta aplicação (incluindo as rotas deste requisito RF-007) são rigorosamente protegidas** pelo motor de identidade consolidado no MVP.
>
> Uma simples requisição GET/POST direta aos endpoints via navegador ou curl, sem o contexto de autenticação, resultará em bloqueio liminar (`401 Unauthorized` ou `403 Forbidden`). **Isso não configura falha na API, mas sim o pleno e correto funcionamento do controle de acesso.**
>
> A aplicação utiliza tokens de Sessão Opaca (UUID) gerados localmente pela própria infraestrutura para controle e exibição de dados. Caso o avaliador necessite de credenciais ou mocks detalhados para testar o bypass, os testes práticos de injeção de sessão e os "mocks de acesso" estão centralizados e detalhados na documentação de fundação de segurança: **`/docs/requisitos/RF-001-gestao-identidade.md`** ou no script `database/seeds/rf-001-gestao-identidade-seeds.sql`.

Interface com CRUD de rastreamento de contatos disponível em: https://sistema-erp-li-3-facsenac.vercel.app/

Operações Implementadas (CRUD do Módulo de Contatos):

Operação	Método	Endpoint	Status Esperado
Create	POST	/api/orcamentos/{id}/contato	200 OK / 201 Created
Read	GET	/api/orcamentos/pendentes-contato	200 OK
Update	PUT	/api/orcamentos/{id}/contato	200 OK
Delete	DELETE	/api/orcamentos/{id}/contato	204 No Content

Documentação Obrigatória da API: O arquivo de documentação técnica dos endpoints acima está disponível no repositório em: `docs/api/swagger.json`

O arquivo OpenAPI reflete rigorosamente os campos de requestBody e os códigos HTTP (200, 404, 409) testados em ambiente de produção, atendendo aos requisitos do Tópico 4.

---

5. Arquitetura e Decisões

Componentes da Arquitetura

- **Frontend:** HTML5/CSS3/JavaScript (ES6). Manipulação de DOM para gerar deep links e detectar window focus.
- **Backend:** Python com FastAPI. Camadas de Service e Repository isoladas.
- **Banco de Dados:** PostgreSQL hospedado no Supabase, utilizando tabelas associativas e restrições de integridade (FK/UNIQUE).

Decisões Técnicas (ADRs)

ADR-001: Arquitetura de Tabela Associativa

- **Razão:** Em vez de poluir a tabela financeira de orçamentos com colunas de tracking comercial, optou-se por criar a tabela relacional orcamentos_contatos. Isso garante um schema limpo, separando domínios (Financeiro vs. CRM) e permitindo expansões futuras de auditoria sem travar tabelas vitais.

ADR-002: Integração Deep Link (wa.me) no Client-Side

- **Razão:** Utilizar a API Cloud Oficial da Meta demandaria custos financeiros e configuração de webhooks fora do escopo do MVP. A montagem da URI dinamicamente no frontend resolve a necessidade de negócio com custo zero e zero latência de servidor.

ADR-003: Filtro Lógico no Python (Application-Level Join)

- **Razão:** Optou-se por realizar o cruzamento de orçamentos pendentes com orçamentos contatados diretamente na camada Service do Python, ao invés de uma view complexa no PostgREST. Isso simplifica a manutenção imediata, dadas as permissões estritas de RLS ativas no banco.

ADR-004: Injeção de Identidade via Token JWT/UUID

- **Razão:** Para garantir integridade máxima na auditoria comercial, a rota de POST não possui o campo vendedor_id no seu Payload JSON. A arquitetura força o framework FastAPI a descompactar a identidade a partir do Header de Autorização, eliminando vulnerabilidades de falsidade ideológica.

---

6. Segurança OWASP

Controle 1: Insecure Direct Object Reference (IDOR) - A01:2021

Vulnerabilidade OWASP (Risco): Um operador com conhecimento de ferramentas de rede poderia interceptar a requisição e forçar o envio do POST de contato para o ID de um orçamento que pertence a outro vendedor ou que já foi faturado e pago, corrompendo a integridade dos dados e do fluxo comercial.

Implementação (Como você protege?): Antes de inserir o registro, a camada Service no backend realiza uma busca obrigatória pelo estado atual do objeto no banco de dados. A lógica valida o atributo de status. Se a resposta da base não for estritamente "PENDENTE", a operação é abortada instantaneamente com o lançamento de uma exceção HTTP 409 (Conflict).

Teste (Como provou que funciona?): Submeteu-se uma requisição POST utilizando uma ferramenta de linha de comando apontada para um ID de orçamento já convertido em Ordem de Serviço. A resposta do servidor foi 409 Conflict, com a mensagem "Operação inválida: orçamento já convertido em venda". Nenhum registro indevido foi gravado na tabela associativa.

Controle 2: Insecure Design (Falsificação de Autoria) - A04:2021

Vulnerabilidade OWASP (Risco): Se a API confiasse cegamente nos dados enviados pelo navegador, um usuário mal intencionado poderia modificar o Payload JSON da requisição, inserindo o ID de um colega no campo vendedor_id para manipular as métricas de produtividade da equipe comercial.

Implementação (Como você protege?): O schema Pydantic utilizado na rota de criação não expõe nem aceita a chave de identidade do vendedor no corpo da requisição web. A identidade é declarada como parâmetro de dependência interna do FastAPI, que extrai, decodifica e valida criptograficamente o ID do ator responsável diretamente a partir do Token de Autorização presente nos cabeçalhos HTTP.

Teste (Como provou que funciona?): Ao enviar uma requisição POST forçando a presença do campo vendedor_id no corpo JSON com um UUID de terceiros, o sistema ignorou silenciosamente o dado malicioso e gravou na tabela de contatos do banco de dados o ID exato que assinou o token HTTP enviado, mantendo a auditoria 100% íntegra.

Controle 3: Cross-Site Scripting (XSS) Prevention - A07:2021

Vulnerabilidade OWASP (Risco): Dados não higienizados provenientes de cadastros de terceiros (nomes de clientes ou descrições) contendo caracteres maliciosos poderiam ser interpolados na montagem do link. Ao clicar, o navegador do usuário poderia interpretar a URL formatada incorretamente ou executar carga arbitrária em ambiente web.

Implementação (Como você protege?): O sistema proíbe o envio de variáveis brutas do banco diretamente para protocolos URI. Toda variável de texto inserida na montagem do link wa.me é encapsulada obrigatoriamente pela função nativa encodeURIComponent() do JavaScript, que sanitiza caracteres especiais, convertendo espaços e símbolos em códigos hexadecimais inofensivos para a barra de endereços.

Teste (Como provou que funciona?): Alterou-se o nome de um cliente no banco de dados para incluir os caracteres `<script>alert(1)</script>`. Ao acionar o botão na interface, o sistema gerou a URL convertendo os caracteres de tag para suas equivalências seguras (ex: %3Cscript%3E). O navegador processou o link apenas como texto simples para o aplicativo de mensagens, sem executar nenhum comando indesejado na aba local.